

The Complete Cloud Computing Software Architecture Patterns

By Michael Pogrebinsky



Copyright Notice

The workbook's contents, including (but not limited to) all written material and images, are protected under international copyright and trademark laws.

Any redistribution or reproduction of part or all of the contents in any form is prohibited.

You may not, except with written permission from the author, distribute or commercially exploit the content.

Nor may you transmit it or store it on any other website, forum, or other forms of electronic retrieval systems.

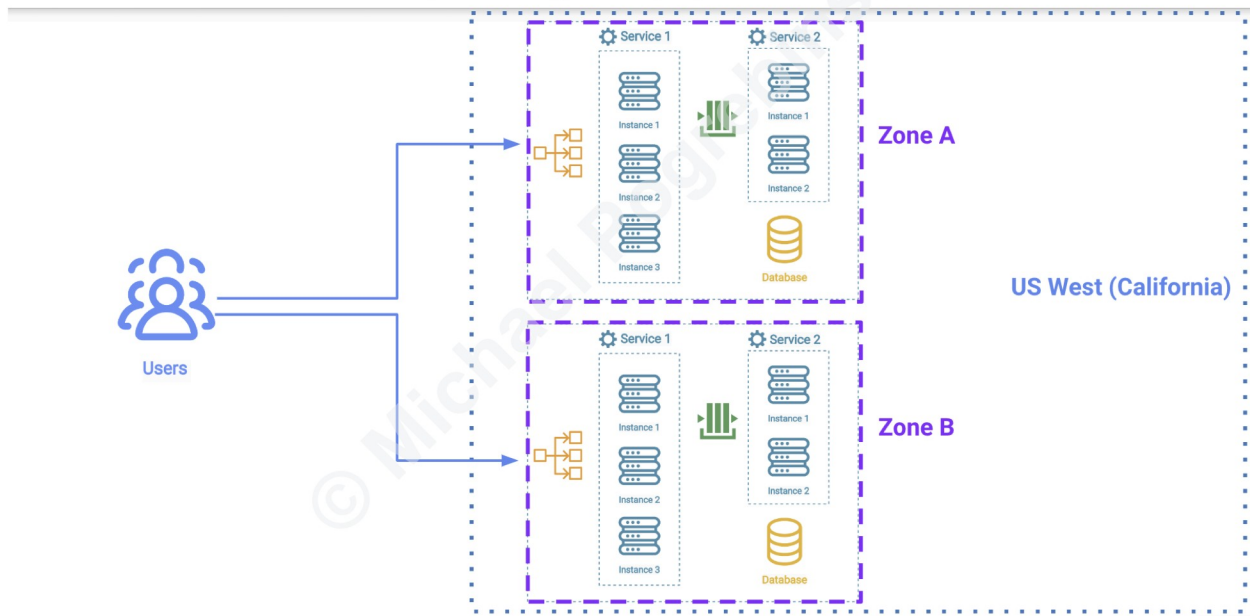
You may print or download to local hard disk extracts for your personal and non-commercial use only.

Introduction	2
Introduction to Cloud Computing Software Architecture Patterns	2
Scalability Patterns	3
Load Balancing Pattern	3
Pipes and Filters Pattern	4
Scatter Gather Pattern	5
Execution Orchestrator Pattern	6
Choreography Pattern	7
Performance Patterns for Data-Intensive Systems	8
Map Reduce Pattern	8
The Saga Pattern	9
Transactional Outbox Pattern	10
Materialized View Pattern	11
CQRS Pattern	12
CQRS + Materialized View for Microservices Architecture	13
Event Sourcing Pattern	14
Software Extensibility Architecture Patterns	15
Sidecar & Ambassador Pattern	15
Anti-Corruption Adapter Pattern	16
Backends for Frontends Pattern	17
Reliability, Error Handling and Recovery Software Architecture Patterns	18
Throttling and Rate Limiting Pattern	18
Retry Pattern	19
Circuit Breaker	20
Dead Letter Queue (DLQ)	21
Deployment and Production Testing Patterns	22
Deployment Patterns:	22
Production Testing Patterns	24

Introduction

Introduction to Cloud Computing Software Architecture Patterns

- **Software Architecture Patterns:**
 - Universal Solutions to Common Design Problems
 - Successfully used by many companies
 - Implementation independent
- **Cloud Computing Benefits:**
 - Access to “infinite” Infrastructure (IaaS)
 - Convenient Pricing Model
 - Tools and Features:
 - FaaS
 - Multi-zone, Multi-Region Deployment

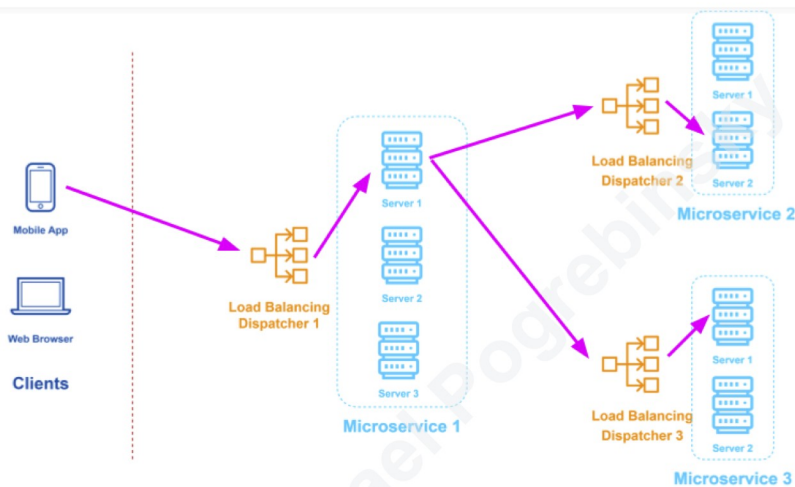


Notes:

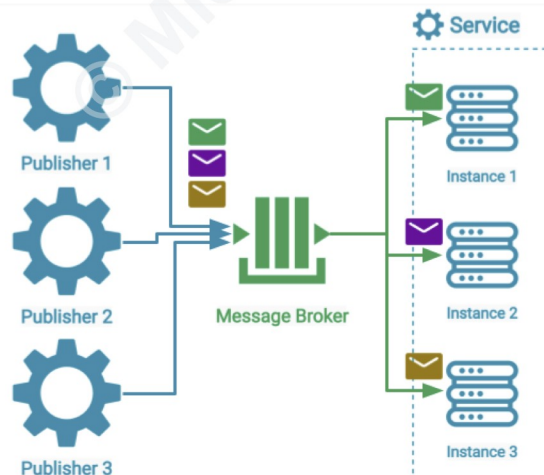
Scalability Patterns

Load Balancing Pattern

- **Goal:** Helps us take advantage of “infinite” access to cloud computers
- **Components:**
 - Source
 - Dispatcher
 - Workers
- **Techniques:**
 - Load Balancing Service



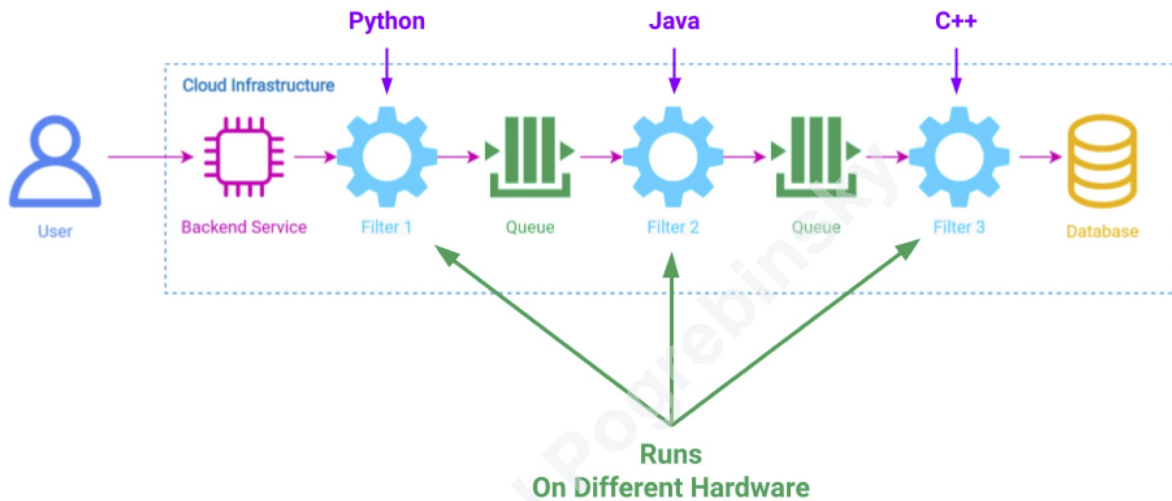
- Message Broker



Notes:

Pipes and Filters Pattern

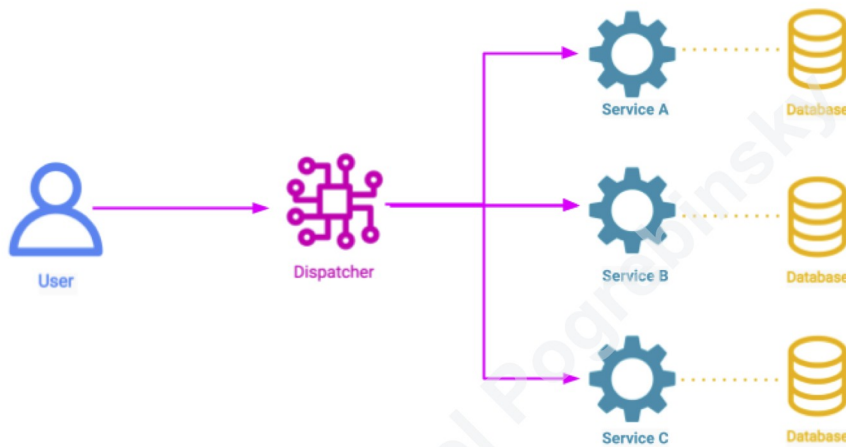
- **Definitions:**
 - Data Source - The origin of the incoming data
 - Data Sink - The final destination of the data
- **Benefits:**
 - Use of a different programming language for each operation
 - Optimal Performance/Cost
 - High Scalability
 - Parallel Execution of tasks



Notes:

Scatter Gather Pattern

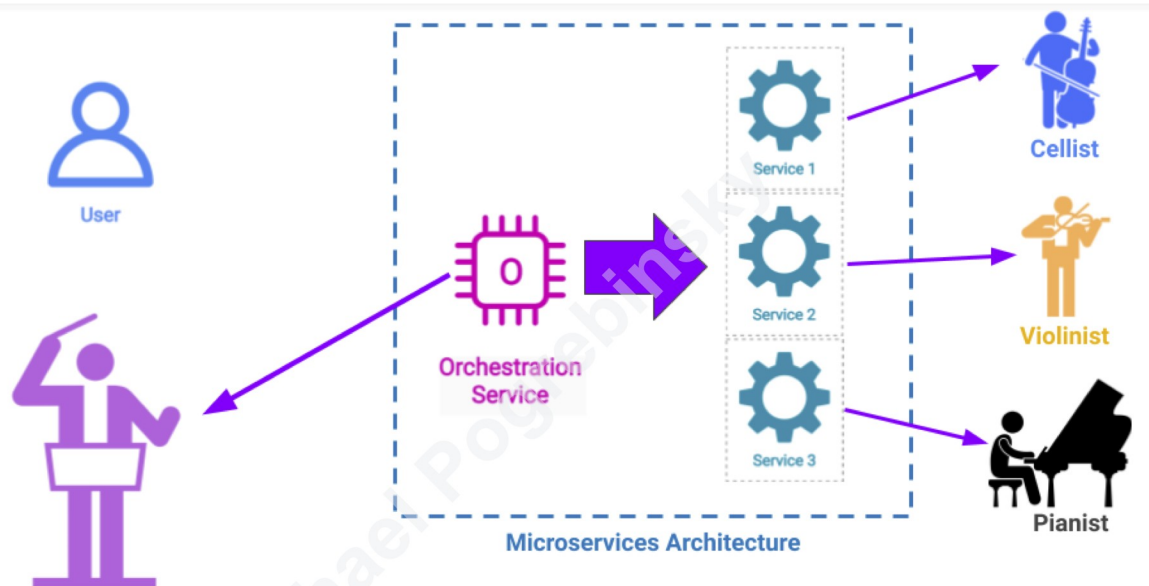
- **Components:**
 - User
 - Dispatcher
 - Workers (different from each other)
- **Core Principles:**
 - Each request to a worker is independent of the other
 - Parallel execution of many requests
 - Providing information in constant time



Notes:

Execution Orchestrator Pattern

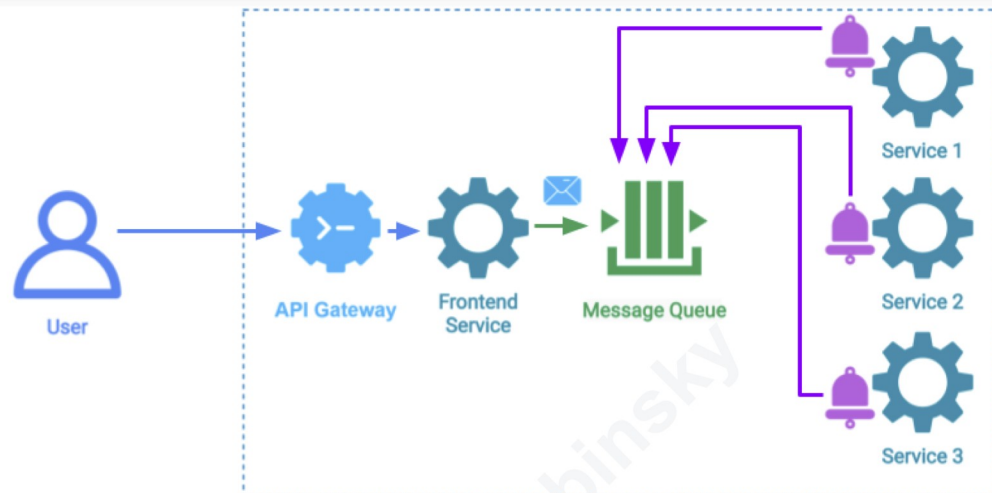
- **Problem:**
 - Running a complex flow of operations across multiple service/microservices
- **Solution:**
 - A centralized service acting as the “brain.”
 - The only service aware of the
 - Context
 - Execution Steps
 - Responsible for handling issues and retries



Notes:

Choreography Pattern

- **Problems:**
 - “Run a flow of business operations that involve multiple services.”
 - Tight coupling between microservices using Orchestration Pattern
- **Solution:** Event-Driven Architecture using Choreography:

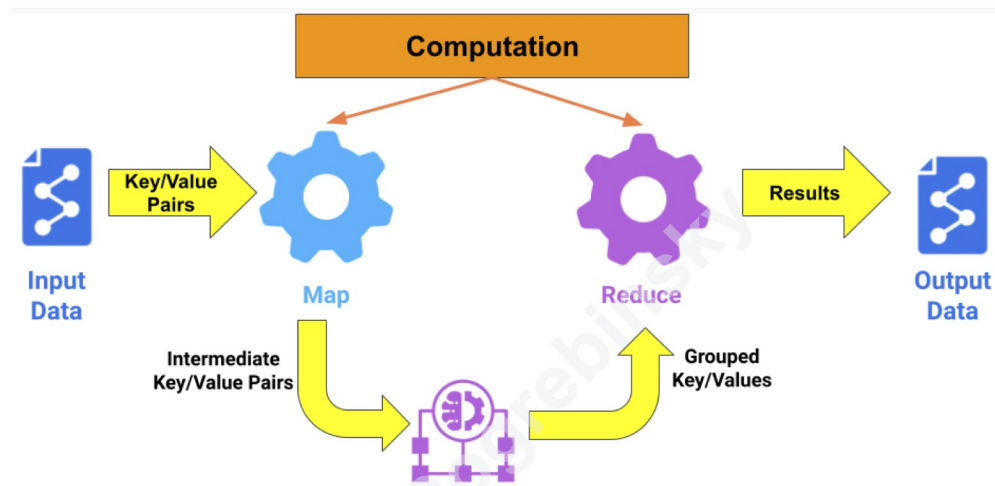


Notes:

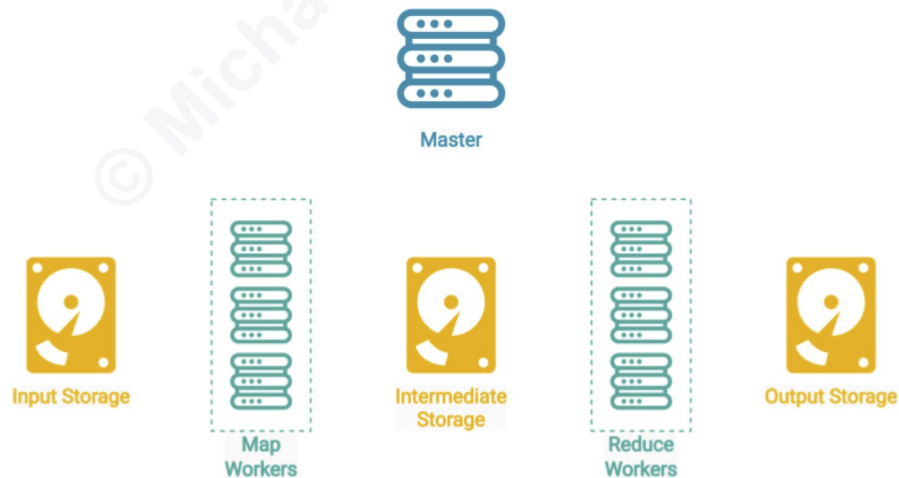
Performance Patterns for Data-Intensive Systems

Map Reduce Pattern

- **Problem:**
 - The complexity involved in running computations on very large inputs
- **Solution:**
 - Follow the same programming model: Map-Reduce



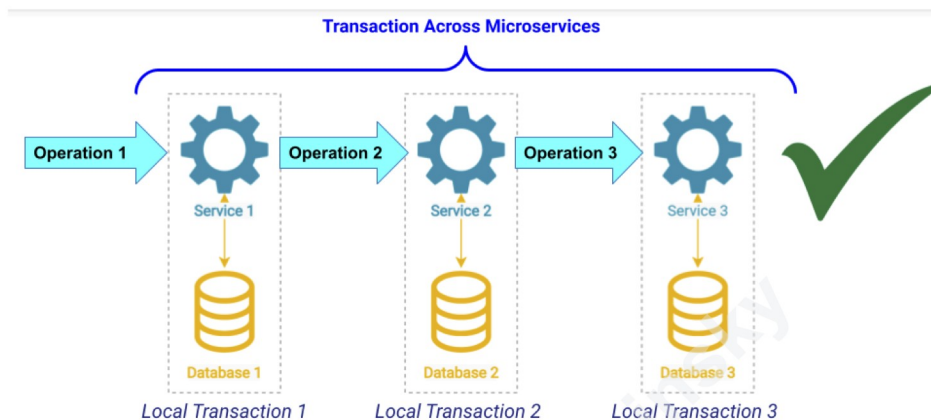
- Reuse the execution framework/architecture



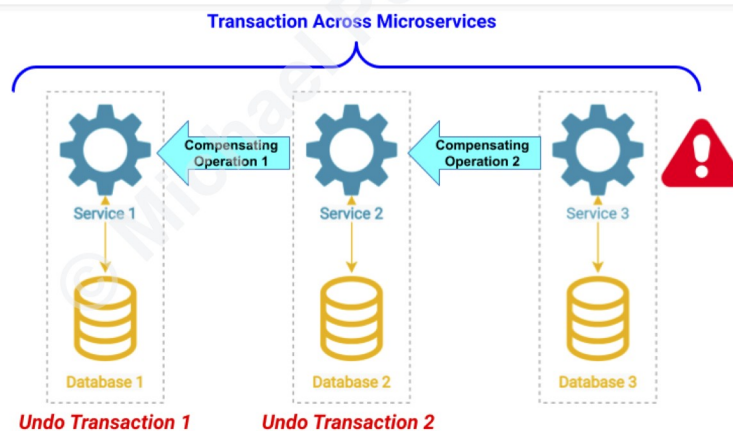
Notes:

The Saga Pattern

- **Problem:**
 - “Lose of ACID Transaction properties in a distributed transaction across multiple microservices.”
- **Solution:**
 - Series of local transactions on success



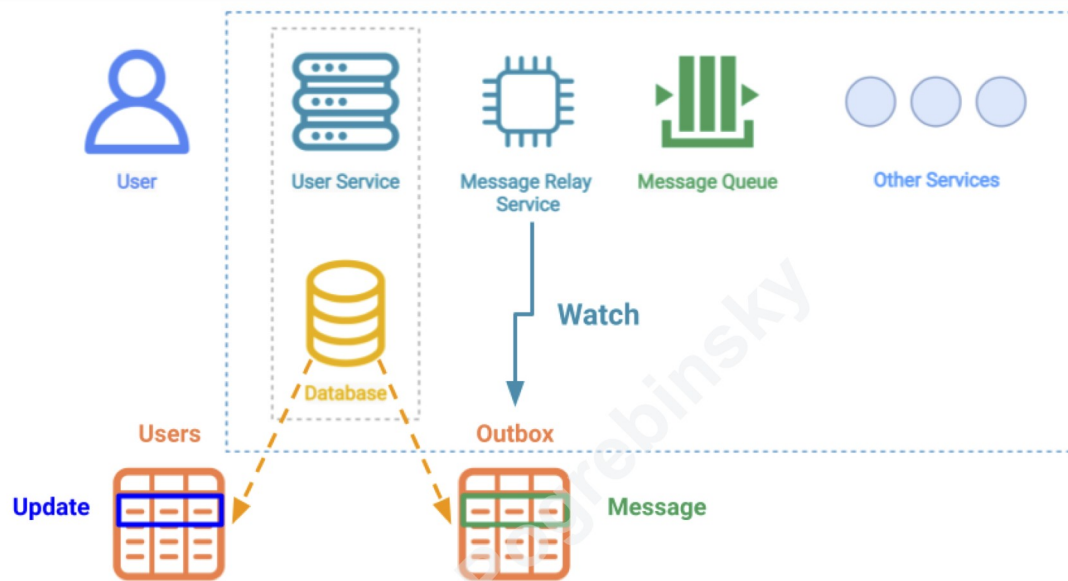
- Compensating operations to abort upon failure



Notes:

Transactional Outbox Pattern

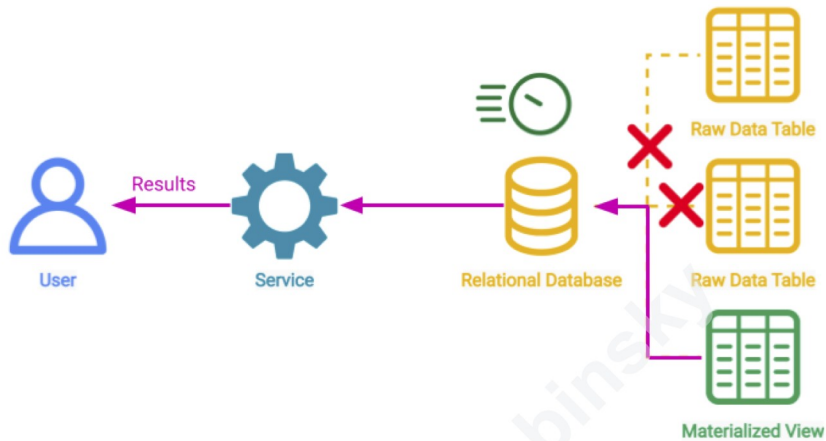
- **Problem:**
 - Database operations and event publishing cannot be part of a single transaction
- **Solution:**
 - Transactional Outbox Table
 - Message Relay Service



Notes:

Materialized View Pattern

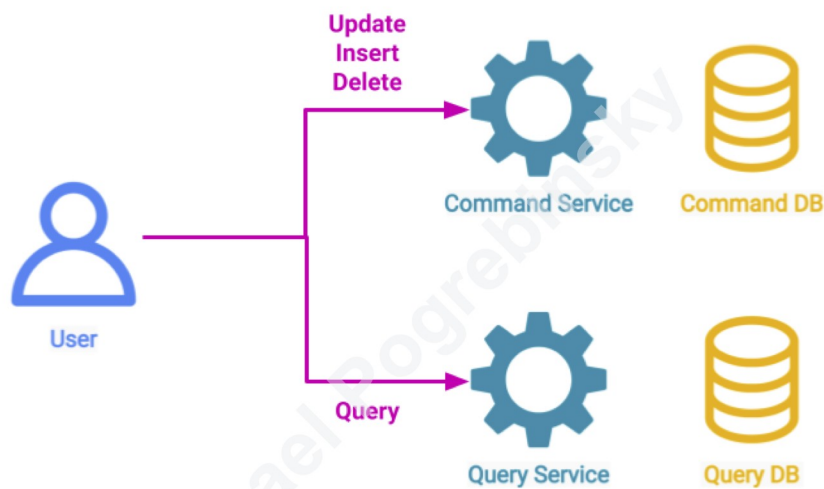
- **Problem:**
 - Low Efficiency and Performance of key database queries
- **Solution:**
 - Read-only Materialized View Table, containing a long query result



Notes:

CQRS Pattern

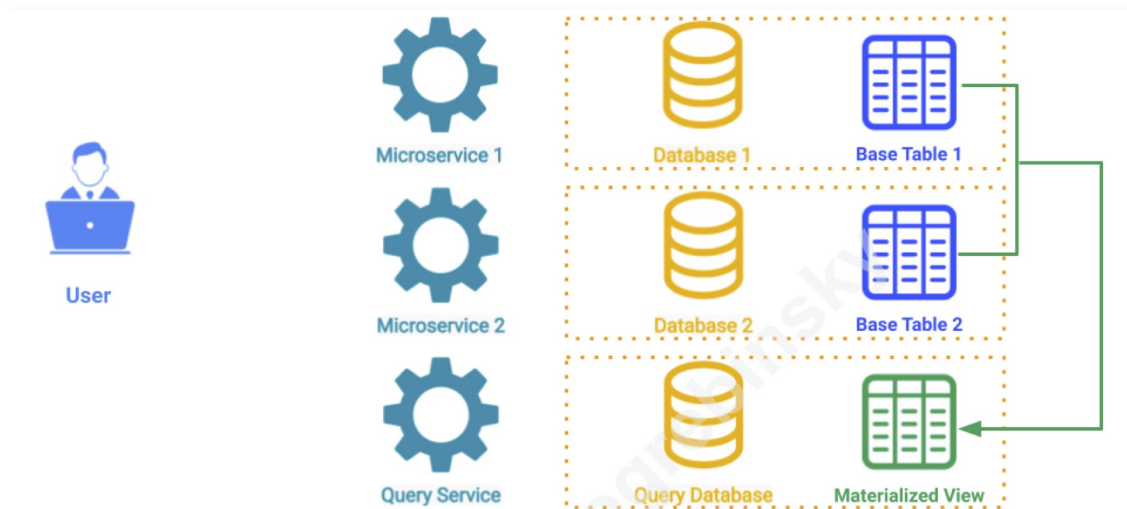
- **Definitions:**
 - CQRS - “Command and Query Responsibility Segregation”
 - Command - Data mutation
 - Query - Data *read* operation
- **Problem:**
 - Tight coupling of Command and Query in one microservice and database
- **Solution:**
 - Separation of Command and Query into separate services and databases



Notes:

CQRS + Materialized View for Microservices Architecture

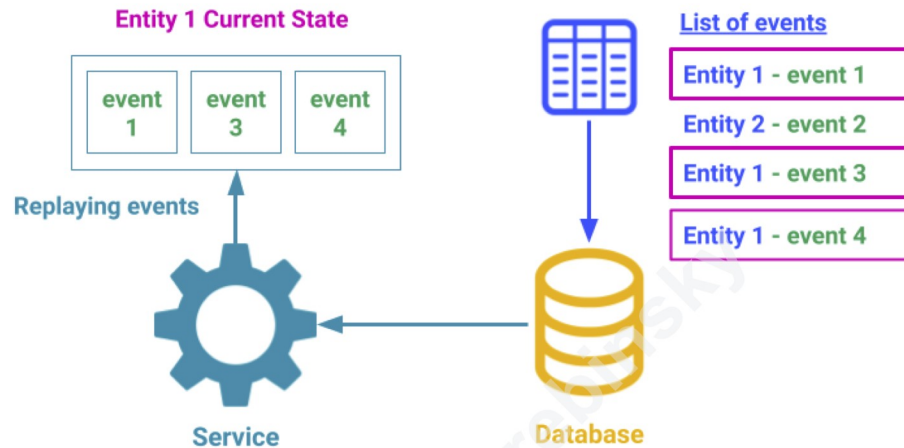
- **Problem:**
 - No “Join” operation for data belonging to different microservices
- **Solution:**
 - CQRS and Materialized View Patterns



Notes:

Event Sourcing Pattern

- **Problem:**
 - CRUD operations overwrite a mutable state.
 - The way we get to a particular state is important.
- **Solution:**
 - Storing the state as a series of events using Event Sourcing

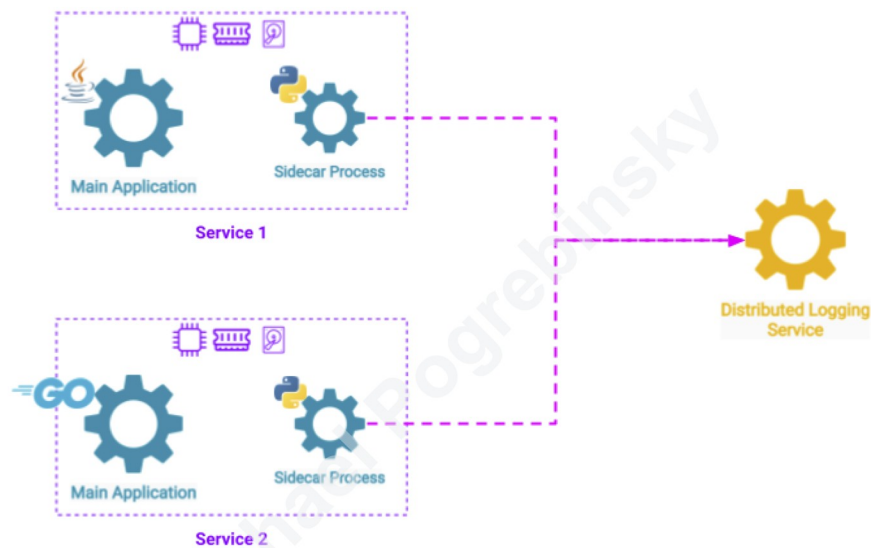


Notes:

Software Extensibility Architecture Patterns

Sidecar & Ambassador Pattern

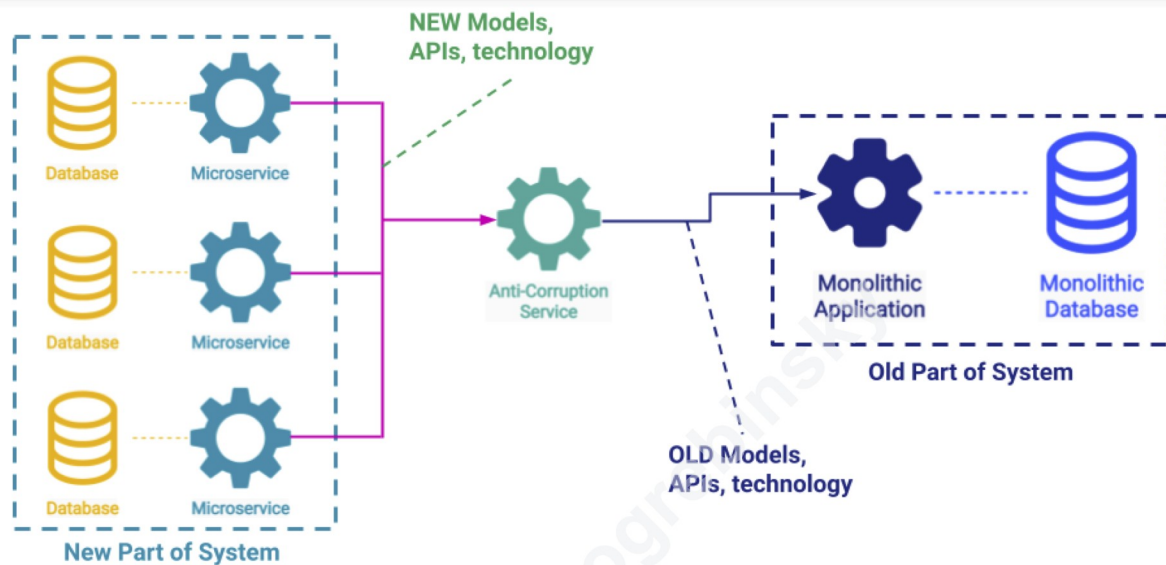
- **Problems:**
 - Many services share common functionality.
 - A shared library restricts technology choice and tightly couples codebases.
- **Solution:**
 - Offload the functionality to a separate process, running on the same host as the main application



Notes:

Anti-Corruption Adapter Pattern

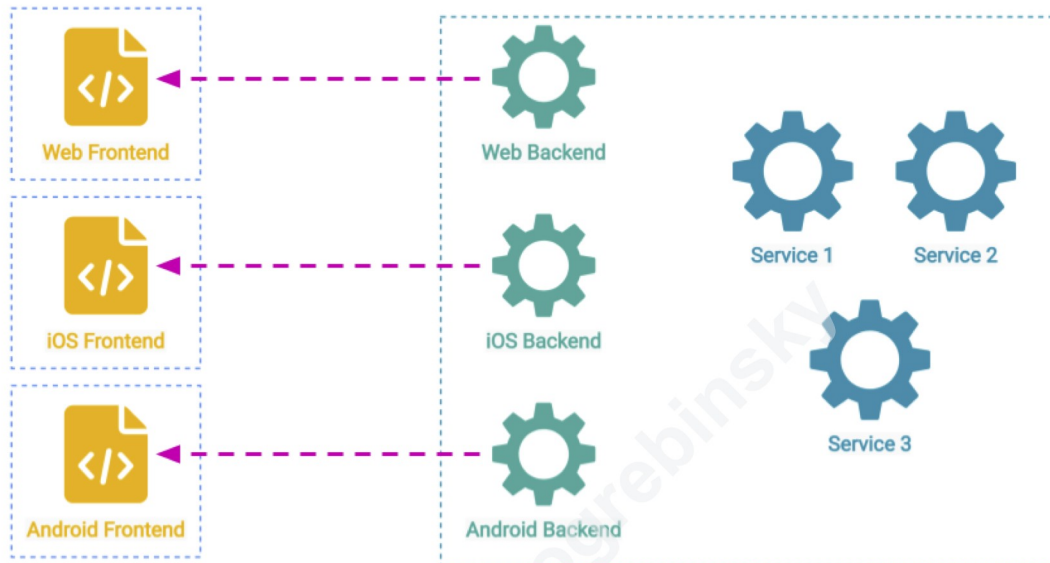
- **Problem:**
 - Corruption of the new part of the system with old API and data models
- **Solution:**
 - Anti-Corruption Adapter Service



Notes:

Backends for Frontends Pattern

- **Problem:**
 - Unified experience in one backend will not take full advantage of each type of frontend
- **Solution:**
 - Separate backend for each type of frontend

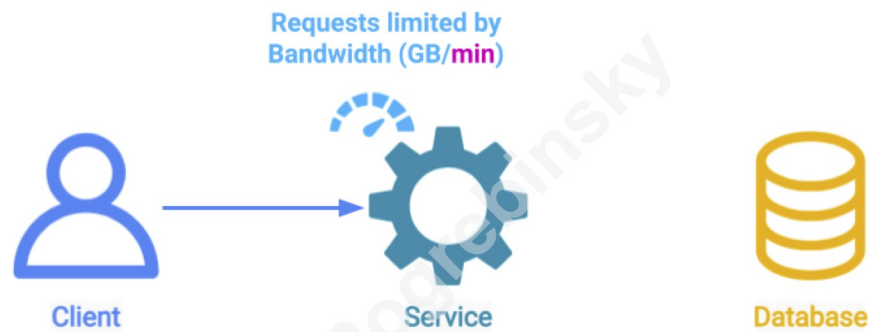


Notes:

Reliability, Error Handling and Recovery Software Architecture Patterns

Throttling and Rate Limiting Pattern

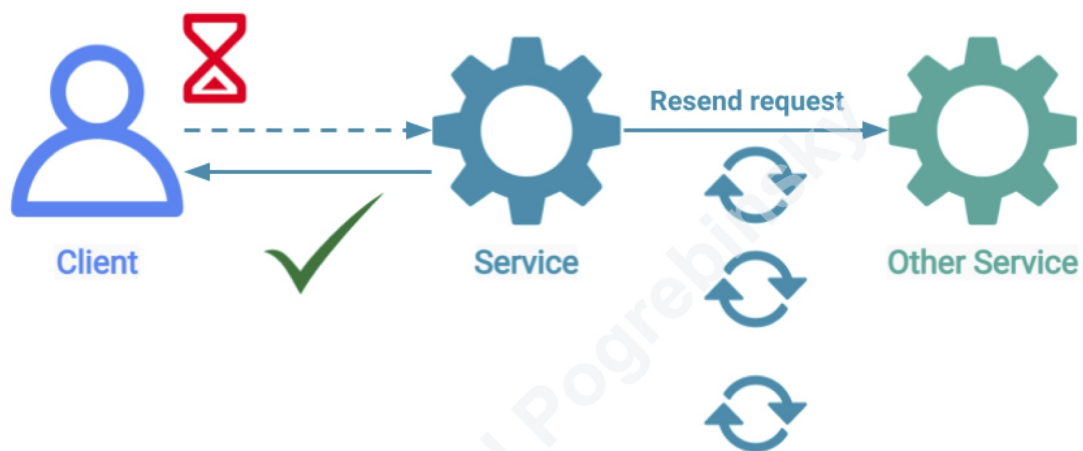
- **Problems:**
 - Potential overconsumption of resources in our system
 - Overconsumption of external APIs
- **Solution:**
 - Set a rate limit on API calls from the client



Notes:

Retry Pattern

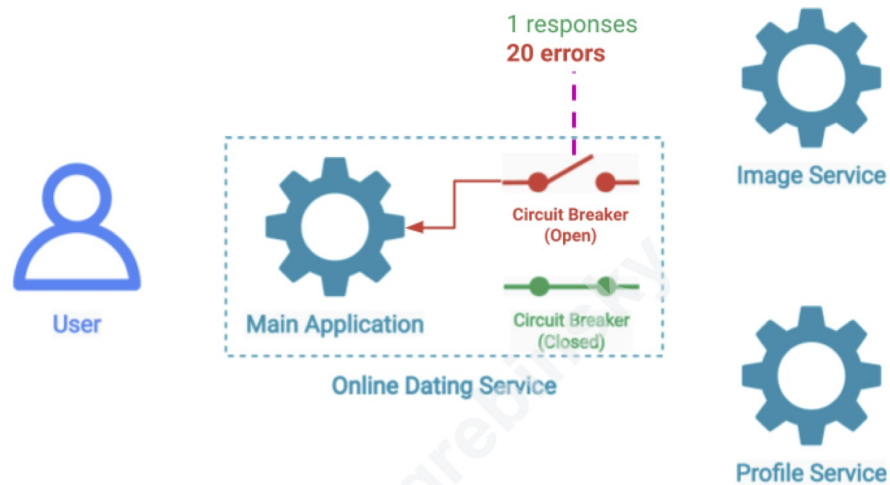
- **Problems:**
 - Internal
 - Software errors
 - Hardware errors
 - Network errors
- **Solution:**
 - Retry the request until successful



Notes:

Circuit Breaker

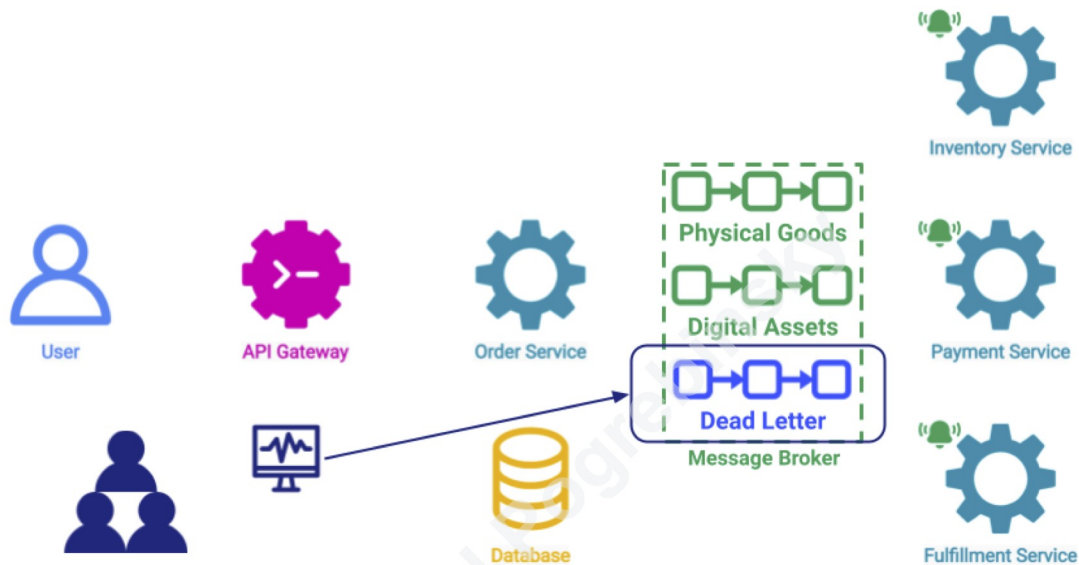
- **Problem:**
 - The last N requests to the Image Service in the last minute failed
 - The error is not short, temporary or recoverable
- **Solution:**
 - One Circuit Breaker per external service



Notes:

Dead Letter Queue (DLQ)

- **Definition:**
 - Special queue in a message broker for messages that cannot be delivered to their destination
- **Types of ways for a message to get into the DLQ:**
 - Programmatic publishing
 - Automatic transfer of messages from original queue

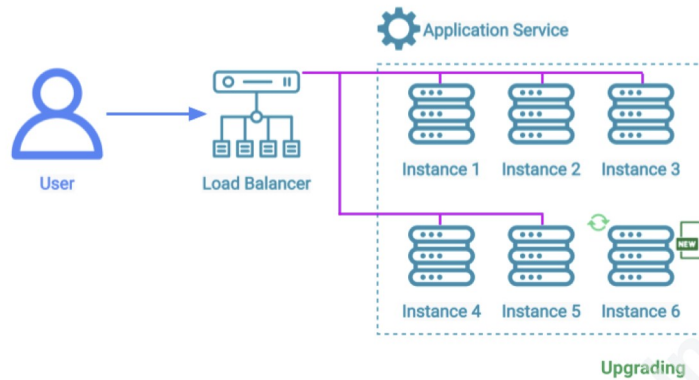


Notes:

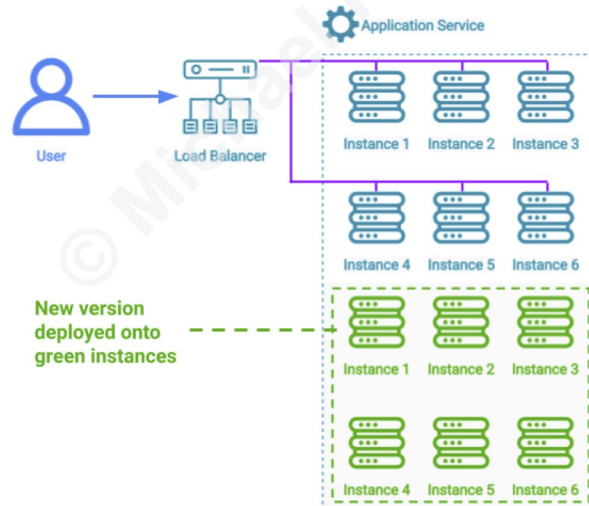
Deployment and Production Testing Patterns

Deployment Patterns:

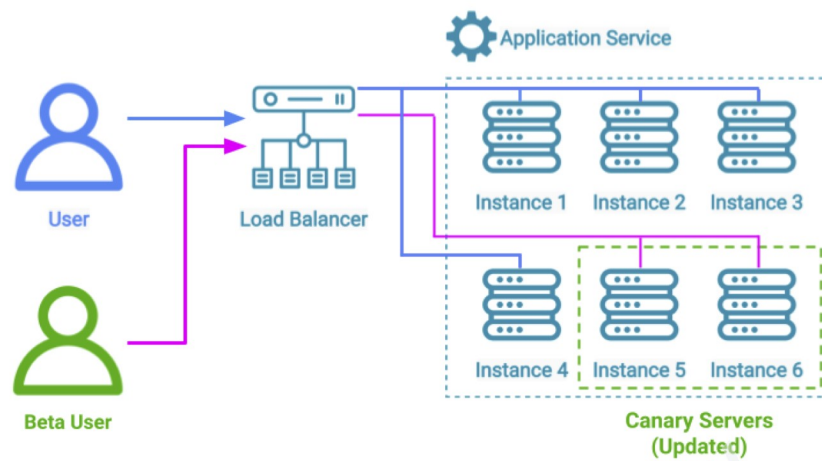
- Rolling Deployment



- Blue-Green Deployment



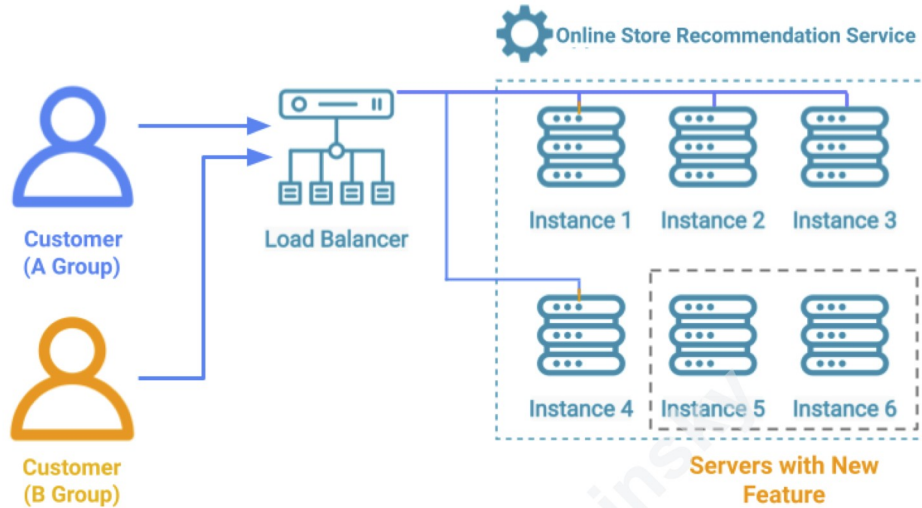
- Canary Release



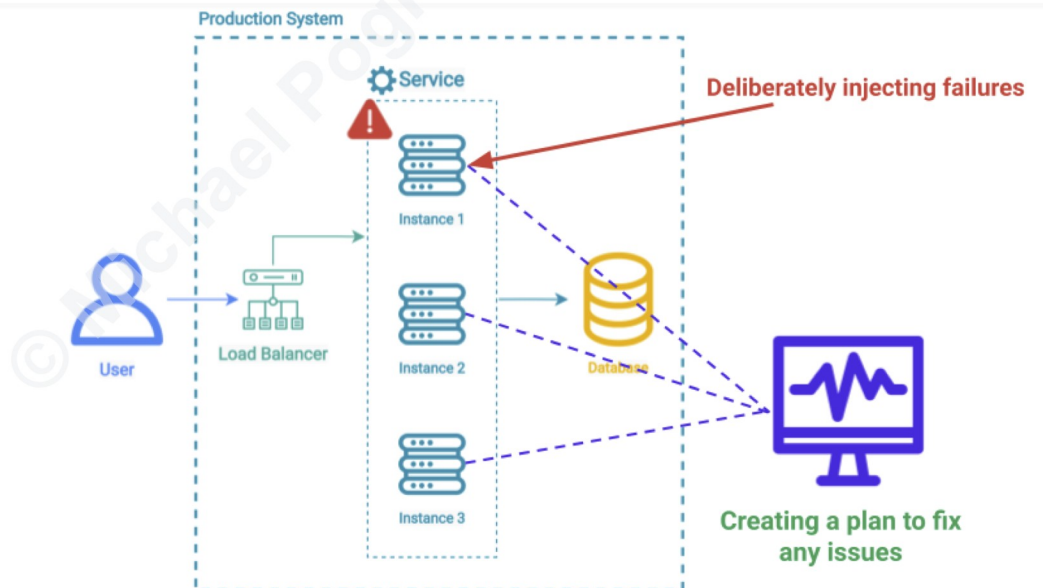
Notes:

Production Testing Patterns

- A/B Testing



- Chaos Engineering Pattern



Notes: